DLP Lab1: Backpropagation

Yi-Hsiang Ho, 111550106

1. Introduction

In this lab, we need to build a Multi-Layer Perceptron (MLP) from scratch using only NumPy, without modern deep learning frameworks. The primary objective is to acquire hands-on understanding of the backpropagation algorithm by explicitly deriving and implementing the forward and backward passes.

The neural network is trained and evaluated on two binary classification tasks: a linearly separable dataset and the non-linearly separable XOR dataset. By constructing a complete training pipeline, including forward propagation, loss computation, and gradient descent, we may demonstrate how multi-layer architectures solve non-linear problems and understand the inner workings of neural network training.

2. Implementation Details

2.1. Sigmoid Function

The sigmoid function squashes inputs into $(0, 1)$ for binary classification probabilities. Its forward pass is:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \tag{1}$$

By the quotient rule, the derivative for the backward pass is:

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x) \cdot (1 - \sigma(x)) \tag{2}$$

The implementation is as follows:

```python
class Sigmoid:
    def __init__(self):
        pass

    def forward(self, x: np.ndarray) -> np.ndarray:
        return 1 / (1 + np.exp(-x))

    def backward(self, x: np.ndarray) -> np.ndarray:
        return self.forward(x) * (1 - self.forward(x))
```

Figure 1. **The implementation of sigmoid funciton class**

2.2. Neural network architecture

The architecture of the neural network can be formulated as:

$$\begin{aligned} z_1 &= \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1, \\ \mathbf{h}_1 &= \sigma(z_1), \\ z_2 &= \mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2, \\ \mathbf{h}_2 &= \sigma(z_2), \\ z_3 &= \mathbf{W}_3 \mathbf{h}_2 + \mathbf{b}_3, \\ y &= \sigma(z_3) \end{aligned} \tag{3}$$

where $\mathbf{W}_1$, $\mathbf{W}_2$, and $\mathbf{W}_3$ are the weight matrices, $\mathbf{b}_1$, $\mathbf{b}_2$, and $\mathbf{b}_3$ are the bias vectors, and σ is the sigmoid function.

Each layer is wrapped into a class. The implementation of linear layer is in Fig. 2. The implementation of the network is in Fig. 3.

```python
class Linear:
    def __init__(self, input_size: int, output_size: int):
        self.weights = np.random.randn(input_size, output_size)
        self.bias = np.random.randn(output_size)

    def forward(self, x: np.ndarray) -> np.ndarray:
        """...
        return np.dot(x, self.weights) + self.bias

    def backward(self, grad: np.ndarray) -> np.ndarray:
        """...
        return np.dot(grad, self.weights.T)

    def update(self, lr: float, input: np.ndarray, grad: np.ndarray):
        self.weights -= lr * np.dot(input.T, grad)
        self.bias -= lr * np.sum(grad, axis=0)
```

Figure 2. **The implementation of linear layer class**

```python
class SimpleNet:
    def __init__(self, input_size: int, hidden_size: int, output_size: int, lr: float = 1e-3):
        self.input_layer = Linear(input_size, hidden_size)
        self.hidden_layer = Linear(hidden_size, hidden_size)
        self.output_layer = Linear(hidden_size, output_size)
        self.sigmoid = Sigmoid()
        self.lr = lr

    def forward(self, x: np.ndarray) -> np.ndarray:
        self.input = x
        self.z1 = self.input_layer.forward(x)
        self.hidden1 = self.sigmoid.forward(self.z1)
        self.z2 = self.hidden_layer.forward(self.hidden1)
        self.hidden2 = self.sigmoid.forward(self.z2)
        self.z3 = self.output_layer.forward(self.hidden2)
        self.output = self.sigmoid.forward(self.z3)
        return self.output
```

Figure 3. **The implementation of neural network architecture**

2.3. Backpropagation

Given the loss gradient with respect to the network output $\frac{\partial L}{\partial y}$, backpropagation computes the gradient for each layer using the chain rule. The gradients with respect to the pre-activations z_3, z_2, and z_1 are computed as follows:

$$\begin{aligned}\frac{\partial L}{\partial z_3} &= \frac{\partial L}{\partial y} \odot \sigma'(z_3), \\ \frac{\partial L}{\partial z_2} &= \left(\frac{\partial L}{\partial z_3}\mathbf{W}_3^T\right) \odot \sigma'(z_2), \\ \frac{\partial L}{\partial z_1} &= \left(\frac{\partial L}{\partial z_2}\mathbf{W}_2^T\right) \odot \sigma'(z_1)\end{aligned} \tag{4}$$

where $\odot$ denotes element-wise multiplication.

With these gradients, the network parameters for each layer $l \in \{1, 2, 3\}$ are updated via gradient descent:

$$\begin{aligned}\mathbf{W}_l &\leftarrow \mathbf{W}_l - \eta\left(\mathbf{h}_{l-1}^T \frac{\partial L}{\partial z_l}\right), \\ \mathbf{b}_l &\leftarrow \mathbf{b}_l - \eta \sum \frac{\partial L}{\partial z_l}\end{aligned} \tag{5}$$

where η is the learning rate, $\mathbf{h}_0 = \mathbf{x}$, and the sum for the bias gradient is taken over the batch dimension.

The implementation of backpropagation and weight update is combined in the implementation of each layer, as shown in Fig. 4 and Fig. 5.

```python
class Linear:
    def __init__(self, input_size: int, output_size: int):...

    def forward(self, x: np.ndarray) -> np.ndarray:...

    def backward(self, grad: np.ndarray) -> np.ndarray:
        """
        Args:
            grad (np.ndarray):  Tensor of shape (batch_size, output_size).
        Returns:
            np.ndarray: Tensor of shape (batch_size, input_size).
        """
        return np.dot(grad, self.weights.T)

    def update(self, lr: float, input: np.ndarray, grad: np.ndarray):
        self.weights -= lr * np.dot(input.T, grad)
        self.bias -= lr * np.sum(grad, axis=0)


class Sigmoid:
    def __init__(self):...

    def forward(self, x: np.ndarray) -> np.ndarray:...

    def backward(self, x: np.ndarray) -> np.ndarray:
        return self.forward(x) * (1 - self.forward(x))
```

Figure 4. **The implementation of backpropagation**

```python
class SimpleNet:
    def __init__(...
    ):...

    def forward(self, x: np.ndarray) -> np.ndarray:...

    def backward(self, grad_loss: np.ndarray):
        self.grad_output = grad_loss * self.sigmoid.backward(self.z3)
        self.grad_hidden2 = self.output_layer.backward(self.grad_output) * self.activation.backward(self.z2)
        self.grad_hidden1 = self.hidden_layer.backward(self.grad_hidden2) * self.activation.backward(self.z1)

    def update(self):
        self.output_layer.update(self.lr, self.hidden2, self.grad_output)
        self.hidden_layer.update(self.lr, self.hidden1, self.grad_hidden2)
        self.input_layer.update(self.lr, self.input, self.grad_hidden1)
```

Figure 5. **The implementation of backpropagation in neural network**

2.4. Others

In my implementation, the network supports batch processing, which means that the network can process multiple samples at once. This is achieved by using numpy's matrix operations. In default, the batch size is the total number of samples, which is 1000.

During inference, the network is used to predict the class of each sample. if the output value is greater than 0.5, the class is 1, otherwise the class is 0.

3. Experimental Results

3.1. Settings

Setting	Value
Learning rate	5×10^{-3}
Hidden layer size	16
Epochs	10000
Number of training data (in linear dataset)	1000
Number of testing data (in linear dataset)	100

3.2. Comparison figure

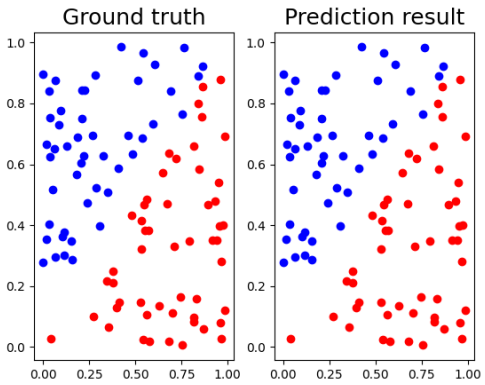


Figure 6. **Comparison figure of linear dataset**

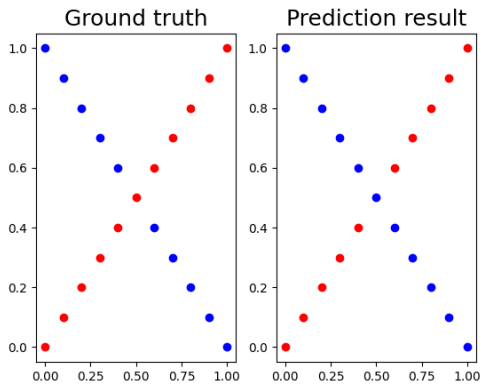


Figure 7. **Comparison figure of XOR dataset**

3.3. Accuracy

```
pwsh> python main.py --data linear
===== Training =====
Epoch 0, Loss: 0.3708191783474561
Epoch 500, Loss: 0.016637882197402257
Epoch 1000, Loss: 0.0109802513916827
Epoch 1500, Loss: 0.009068565663205486
Epoch 2000, Loss: 0.008052513590890662
Epoch 2500, Loss: 0.007390586743521656
Epoch 3000, Loss: 0.0069144981086260784
Epoch 3500, Loss: 0.0065509924999925225
Epoch 4000, Loss: 0.006261764076485862
Epoch 4500, Loss: 0.006024484160193813
Epoch 5000, Loss: 0.005825123932396032
Epoch 5500, Loss: 0.005654362100971763
Epoch 6000, Loss: 0.005505732331739316
Epoch 6500, Loss: 0.005374594392054099
Epoch 7000, Loss: 0.005257527469209096
Epoch 7500, Loss: 0.0051519541961687985
Epoch 8000, Loss: 0.005055897662951063
Epoch 8500, Loss: 0.0049678186099495905
Epoch 9000, Loss: 0.004886502868689085
Epoch 9500, Loss: 0.004810981393197581
===== Testing =====
acc: 100%
```

Figure 8. **Accuracy of linear dataset**

```
pwsh> python main.py --data XOR
===== Training =====
Epoch 0, Loss: 0.417950681649704
Epoch 500, Loss: 0.22878321879470734
Epoch 1000, Loss: 0.20256831203696296
Epoch 1500, Loss: 0.16934392737893944
Epoch 2000, Loss: 0.1337946698300422
Epoch 2500, Loss: 0.10478037393871806
Epoch 3000, Loss: 0.08387212228656972
Epoch 3500, Loss: 0.06891608492845391
Epoch 4000, Loss: 0.057814900883227036
Epoch 4500, Loss: 0.049160773375812594
Epoch 5000, Loss: 0.04210742645304709
Epoch 5500, Loss: 0.03618019018384853
Epoch 6000, Loss: 0.031121527193335212
Epoch 6500, Loss: 0.026786822942808635
Epoch 7000, Loss: 0.023082617064637066
Epoch 7500, Loss: 0.01993450846064279
Epoch 8000, Loss: 0.017273740090774193
Epoch 8500, Loss: 0.015033985412275411
Epoch 9000, Loss: 0.013152427619694354
Epoch 9500, Loss: 0.011571715446743236
===== Testing =====
acc: 100%
```

Figure 9. **Accuracy of XOR dataset**

3.4. Learning Curve

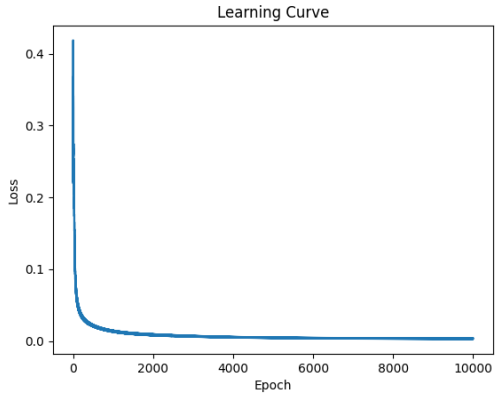


Figure 10. **Learning curve of linear dataset**

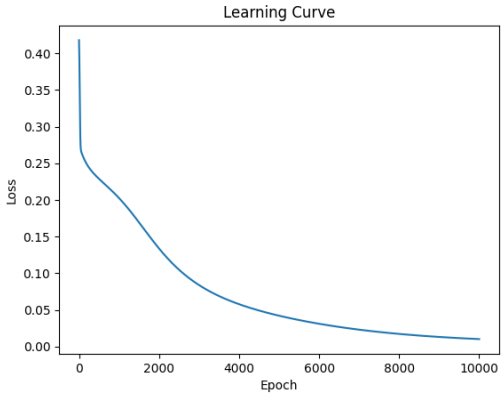


Figure 11. **Learning curve of XOR dataset**

4. Discussion

Note that the experiments below follow the settings in Sec. 3.1, except for the target variable.

4.1. Different Learning Rates

The learning rate (LR) determines the step size during gradient descent. The results are shown in Fig. 12, Fig. 13, Fig. 14, and Fig. 15. From the experiments with $\eta \in \{0.1, 0.01, 0.001, 0.0001\}$:

- **High LR (0.1, 0.01):** The learning process is quite fast. They successfully converge on XOR dataset and gain 100% accuracy. However, in linear dataset, $\eta = 0.01$ still converges fast, but $\eta = 0.1$ fails to converge, staying at 50% accuracy. This occurs because the large learning rate, combined with a batch size of 1000 samples, produces an massive weight update in the initial steps. This magnitude pushes the weights into the saturation region of the Sigmoid function, where the derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ is nearly zero. Consequently, the gradients vanish, and the model's weights stop updating, leaving it stuck in a state of poor performance.

- **Low LR (0.001, 0.0001):** The learning process is much slower. Within 10,000 epochs, both two cases cannot converge on XOR dataset, and the accuracy is 71% and 33%, respectively. On the other hand, in linear dataset, both two cases converge successfully, but in a slower speed.

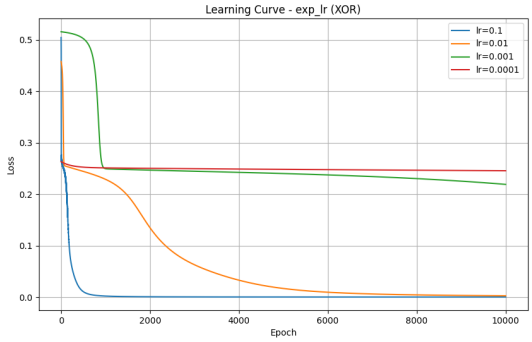


Figure 12. **Learning curves for different learning rates on XOR dataset**

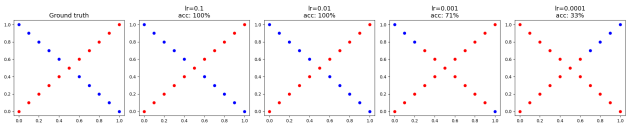


Figure 13. **Prediction results for different learning rates on XOR dataset**

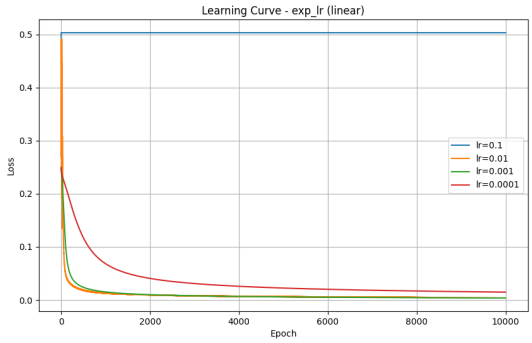


Figure 14. **Learning curves for different learning rates on linear dataset**

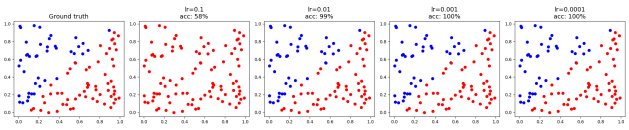


Figure 15. **Prediction results for different learning rates on linear dataset**

4.2. Different Number of Hidden Units

The number of hidden units represents the model's capacity to learn complex patterns. I tested $hs \in$

$\{4, 16, 64, 256\}$, as shown in Fig. 16, Fig. 17, Fig. 18, and Fig. 19:

- **Small capacity ($hs = 4$):** The model has limited representational power. While it still solves the linear and XOR problems, it converges slower than the larger models.
- **Large capacity ($hs = 16, 64, 256$):** These models converge more rapidly in XOR dataset. However, the computational cost per epoch also increases significantly.

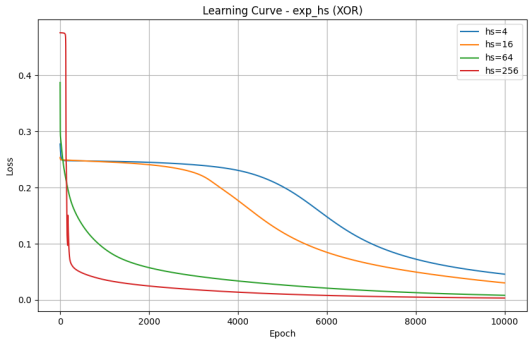


Figure 16. **Learning curves for different number of hidden units on XOR dataset**

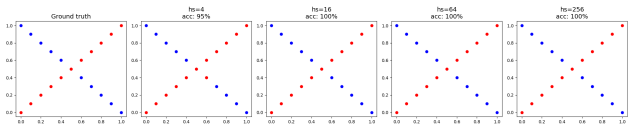


Figure 17. **Prediction results for different number of hidden units on XOR dataset**

However, more hidden units mean more parameters to be learned, which significantly increases the model's complexity. In the linear dataset, we can see that the performance surprisingly drops at $hs = 64$, failing to converge. This is a result of the unscaled random initialization ($\mathcal{N}(0, 1)$) combined with the Sigmoid activation. As the number of hidden units increases, the variance of the pre-activations $z = \mathbf{W}\mathbf{h} + \mathbf{b}$ also increases, pushing the hidden units into the saturation region where the gradient vanishes.

Interestingly, at $hs = 256$, the loss oscillates erratically, but the model eventually manages to converge to 100% accuracy. This suggests that with a sufficiently large number of parameters, the optimization landscape becomes more chaotic, and these oscillations can occasionally "kick" the network out of saturated states, allowing it to eventually find a path to convergence. In contrast, at $hs = 64$, the network remains trapped in a dead state from which it cannot recover within the given 10,000 epochs.

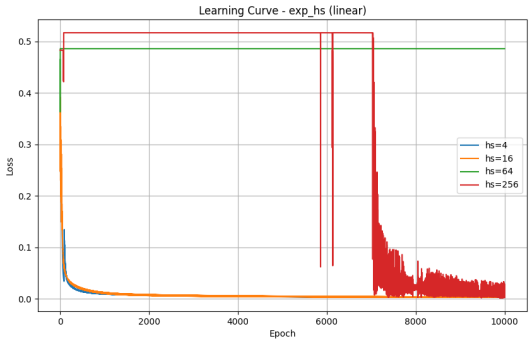


Figure 18. **Learning curves for different number of hidden units on linear dataset**

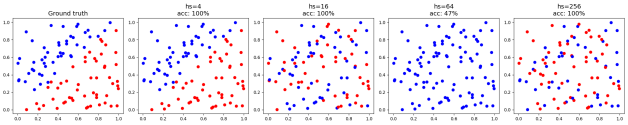


Figure 19. **Prediction results for different number of hidden units on linear dataset**

4.3. Without Activation Functions

Activation functions introduce the non-linearity required to solve the non-linear question such as XOR. As shown in Fig. 20 and Fig. 21, the model fails to converge on XOR dataset without activation functions. In contrast, the model can still solve the linear problem without activation functions (see Fig. 22 and Fig. 23).

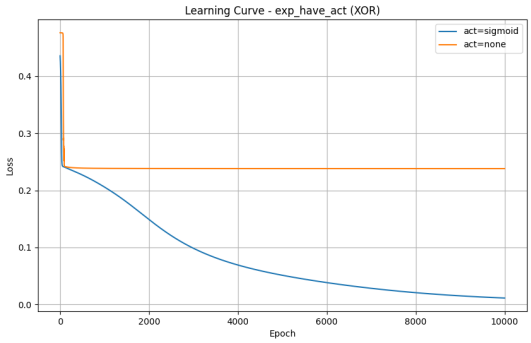


Figure 20. **Learning curves for XOR dataset with and without activation functions**

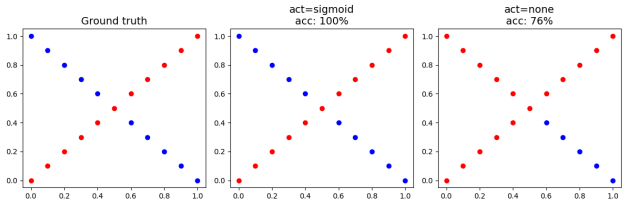


Figure 21. **Prediction results for XOR dataset with and without activation functions**

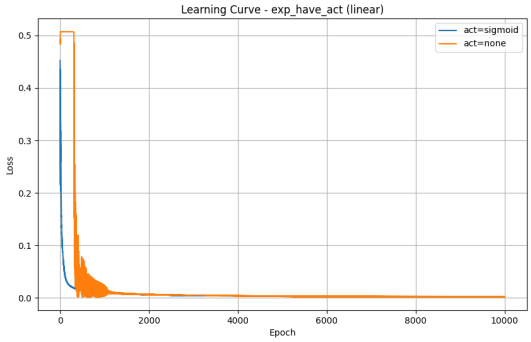


Figure 22. **Learning curves for linear dataset with and without activation functions**

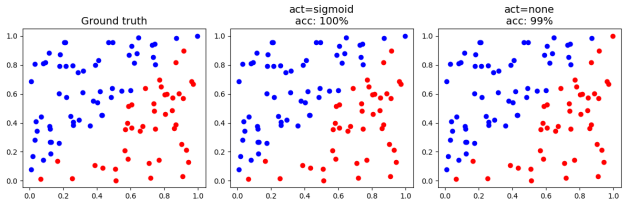


Figure 23. **Prediction results for linear dataset with and without activation functions**

5. Questions

5.1. Question 1

What is the purpose of activation functions?

Ans: Activation functions introduce non-linearity into the neural network. Without them, even a deep network with many layers would just be a sequence of linear transformations, which is equivalent to a single linear model. Non-linear activation functions (like Sigmoid, ReLU, or Tanh) allow the model to learn and represent complex boundaries, such as those required for the non-linearly separable XOR dataset.

5.2. Question 2

What might happen if the learning rate is too large or too small?

Ans: If the learning rate is too large, the weight updates might overshoot the global minimum, causing the loss to oscillate or even diverge. It can also push weights into the saturation regions of activation functions (vanishing gradients), as seen in the $\eta = 0.1$ experiment. If the learning rate is too small, the convergence becomes extremely slow, requiring an impractical number of epochs to minimize the loss, and the model is more likely to get trapped in poor local minima.

5.3. Question 3

What is the purpose of weights and biases in a neural network?

Ans: Weights (W) determine the relative importance or influence of each input connection on the subsequent layer, essentially scaling the signals to learn the underlying patterns. Biases (b) provide an additive offset that allows the activation function to be shifted horizontally. This shifting is crucial because it enables the network to represent boundaries that do not necessarily pass through the origin $(0, 0)$, providing more flexibility in fitting the data.

6. Extra

6.1. Different Activation Functions

I compared `Sigmoid`, `ReLU`, and `Tanh`. Note that these two tasks are both binary classification tasks, so `Sigmoid` activation is used in the output layer no matter which activation function is used internally. The results are shown in Fig. 24, Fig. 25, Fig. 26, and Fig. 27.

- **Sigmoid and Tanh:** Both functions squash the output into a bounded range ($(0, 1)$ and $(-1, 1)$ respectively). They successfully learn the XOR boundary and achieve 100% accuracy. Tanh often converges faster than Sigmoid because its outputs are zero-centered, which aids gradient descent.

- **ReLU:** The model seems to be unstable and sensitive to the initial weights. Although it failed on linear dataset with the same setting as other experiments, I try different random seeds and find that it can converge with some values, as shown in Tab. 1.

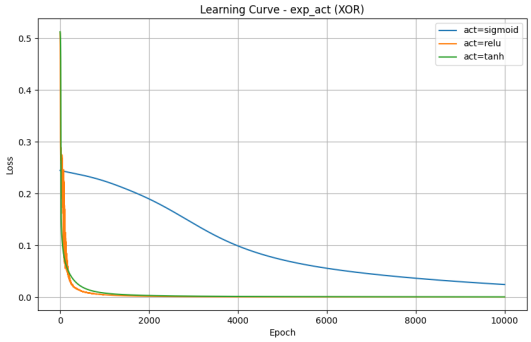


Figure 24. **Learning curves for different activation functions on XOR dataset**

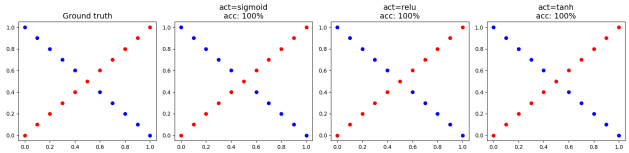


Figure 25. **Prediction results for different activation functions on XOR dataset**

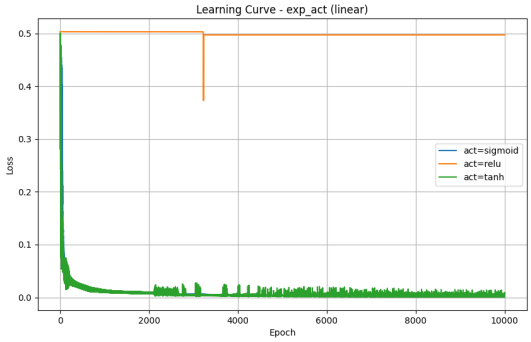


Figure 26. **Learning curves for different activation functions on linear dataset**

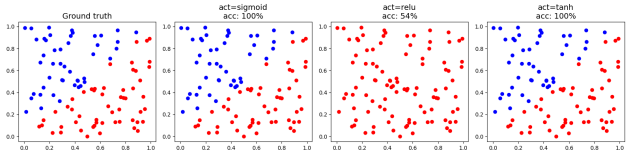


Figure 27. **Prediction results for different activation functions on linear dataset**

seeds	0	10	100	500	1000
acc.	99%	46%	60%	99%	50%

Table 1. **Using different random seeds on linear dataset with ReLU activation function**